

TWILIO · SOLUTIONS ENGINEERING

SURGE

MASTERCARD®

Continental Finance

Twilio Bulk Messaging Demo

Presenter Guide · Step-by-step run of book

Prepared for: Amplix + Continental Finance RFP evaluation
Date: September 11, 2026

Prepared by: Twilio SE

Purpose & flow

This demo answers Continental Finance's SMS RFP by proving three of the four core use cases live against Twilio's Programmable Messaging API. It runs 15–20 minutes end-to-end.

The fourth use case (business-user campaign administration) is addressed as a talk-track in this document — Twilio Console covers most of it honestly, and we concede the one gap explicitly.

THREE USE CASES, ONE BROWSER, ONE CELL PHONE

Use Case 1 Payment reminder — immediate send + scheduled send via Programmable Messaging Create Message.

Use Case 2 Message pull-back — schedule + cancel via Update Message with Status=canceled.

Use Case 3 Response & opt-out — Advanced Opt-Out + free-form capture + DNC.

WHAT YOU'LL ASK THEM TO NOTICE

- Per-message MessageSid — every send is individually addressable, cancellable, and auditable.
- Personalization is server-rendered — full flexibility, no template-language limits.
- Native scheduling from 15 min to 7 days out via scheduleType=fixed + sendAt.
- Cancel-in-flight via Update Message with Status=canceled while message is still scheduled.
- STOP/START/HELP handled natively via Advanced Opt-Out — zero code.
- Free-form inbound captured for human review — real cross-sender DNC.
- At 3.5M/mo (~4/sec average), fan-out fits comfortably inside short-code throughput (100 MPS).

Pre-flight checklist

Complete all six before the demo starts. Total time: ~2 minutes.

STEP 1 Server up on :3001

Terminal 1: `cd ~/Documents/ContinentalFinanceBulkDemo && node server.js`

Watch for: "Continental Finance Bulk Messaging Demo running on http://localhost:3001". No missing env-var warnings.

STEP 2 ngrok tunnel up

Terminal 2: `ngrok http 3001 --url sasmith.ngrok.app`

Browser test: `https://sasmith.ngrok.app/api/queue` should return JSON (may 307 through ngrok abuse page — fine, Twilio's webhook UA bypasses it).

STEP 3 Messaging Service inbound webhook

Console !' Messaging !' Services !' your Surge service !' Integration !' Inbound Webhook URL = `https://sasmith.ngrok.app/webhooks/twilio/inbound`. Method: POST.

STEP 4 Advanced Opt-Out configured

Same service !' Opt-Out Management !' Advanced enabled. Custom Surge STOP copy set (see appendix for suggested wording). Keywords: STOP/START/HELP.

STEP 5 Dashboard open

Browser: `http://localhost:3001` — you should see the Continental Finance SMS Operations header and three Use Case tabs (1, 2, 3).

STEP 6 Cell in hand, signal confirmed

You'll text to the Surge long code from your cell during Use Case 3. Test with a single "test" text before the demo starts — it will show up in the Inbound stream and you'll know the round-trip works.

Fallback: if ngrok flakes or Twilio 5xx during the demo, you have a screen-recorded backup take. Do not attempt to fix live — pivot to the recording and keep talking.

Use Case 1 · Payment Reminder

RFP use case 1: high-volume payment reminder + past-due messaging. Target: 3 minutes.

Open on Use Case 1 tab

Say: "Continental sends 3.5 million payment reminders a month today — both proactive statement-due nudges and past-due delinquency messaging — from short codes, in an 8-to-9 window. Watch what one Programmable Messaging API call looks like."

STEP 1 Click "Send now"

Dashboard hits our server. Server renders Jane's personalized body and calls `client.messages.create()` with the Messaging Service SID. Twilio returns 201 Created + a `MessageSid`. The status card flips through `queued !' sending !' sent !' delivered` as `GET /Messages/{Sid}` is polled every 750ms. Your cell buzzes with the real message.

Say: "One API call, one MessageSid. At Continental Finance's 3.5 million a month — about four per second average — that becomes a fan-out. One call per cardholder. A single short code handles 100 messages per second, so throughput is not the bottleneck. And every message has its own SID: fully cancellable, fully auditable, one row per cardholder in Console."

STEP 2 Click "Schedule for 10am tomorrow ET"

Same Programmable Messaging endpoint, this time with `scheduleType=fixed` and `sendAt=<tomorrow 10:00 ET>`. Twilio returns a `MessageSid` with `status=scheduled`. This message will hold on Twilio's schedule and fire at the `sendAt` time.

Say: "Programmable Messaging supports scheduling from fifteen minutes to seven days out. Continental Finance would compute the per-cardholder sendAt in their scheduler — for a national footprint that means one API call per cardholder with their own local sendAt. The trade-off is honest: no auto-timezone-localization like Bulk offers, but full per-message cancellability."

Talking point

For any scheduled message, Continental Finance can pull it back before send fires by calling `Update Message` with `Status=canceled`. That is exactly what Use Case 2 demonstrates — the same `MessageSid` pattern, extended to the payment-triggered cancel flow.

Use Case 2 · Message Pull-back (Payment-Triggered Cancel)

RFP use case 2: "receive payment/account updates quickly enough to remove customers from active campaigns and prevent unnecessary reminders after payment." Target: 4 minutes.

Open Use Case 2 tab

Say: "Continental Finance's current lag from payment to suppression is about fifteen minutes. Payment posts, but the SMS provider does not know about it in time, and the reminder still fires. Watch how message pull-back on Programmable Messaging closes that gap for scheduled sends."

STEP 1 Click "Schedule Jane's reminder (20 min from now)"

Our server calls POST /2010-04-01/Accounts/{Sid}/Messages.json with scheduleType=fixed, sendAt=+20min, and the Messaging Service SID. Twilio returns a MessageSid and status=scheduled. Jane's row in the queue table shows the MessageSid; status badge flips to "Scheduled" (blue). The Twilio Console link lights up on the right so you can verify Twilio's view of the state.

Say: "Twilio now holds this message on its schedule. It will fire in twenty minutes unless we cancel it first. Every cardholder scheduled the same way gets their own MessageSid — no batch handle, no shared operationId. Each message is independently addressable."

STEP 2 Click "Payment posted !" cancel scheduled SMS"

Simulates Continental Finance's payment system firing a webhook — POST /webhooks/payment with {phone, accountId, amount, postedAt}. Our server looks up Jane's MessageSid, then calls POST /2010-04-01/Accounts/{Sid}/Messages/{MessageSid}.json with Status=canceled. Twilio returns status=canceled. The queue row flips to "Canceled" (red).

Say: "That was one API call to pull back one specific message. Update Message with Status=canceled. Twilio drops the scheduled SMS before it ever reaches the carrier. At Continental Finance's scale, this is the pattern: payment webhook fires, server looks up the MessageSid for that cardholder, cancel API call goes out. Millisecond latency, per-cardholder precision."

STEP 3 Verify in Twilio Console

Click the "Open in Twilio Console" link on the right side of the scene. Console !' Monitor !' Logs !' Messaging shows the MessageSid with status=canceled. Refresh to confirm.

Say: "That is the audit record. Continental Finance's compliance team, or Amplix during their review, can walk this trail for any message: scheduled at X, canceled at Y, never delivered. No cardholder ever saw the past-due reminder they had already paid."

Honest note on the cancellation window

Cancellation is only available while status = scheduled. Twilio moves scheduled messages to queued approximately 15 minutes before sendAt. After that, Update Message returns error 30409 and the message is committed.

Practical implication for Continental Finance: design schedule lead time so there is always a cancel window before send. If a payment can post right up to the reminder fire time, schedule at least 15 minutes into the future.

Say: "This is documented behavior. If evaluators ask what happens if payment posts fourteen minutes before send: Twilio returns error thirty-thousand-four-oh-nine, the message is already committed to

queue. Design your lead time accordingly."

Use Case 3 · Response & Opt-Out

RFP use case 3: capture every inbound, handle STOP compliantly, review the rest, block on request. Target: 5 minutes. Two things happen here — the automated path (STOP/START/HELP handled entirely by Twilio) and the manual path (free-form text captured for a human).

Open Use Case 3 tab

Say: "Every SMS platform has to handle inbound messages from customers. There are two flavors: the compliance-critical keywords like STOP, and everything else. Watch how Twilio handles each."

STEP 1 PART A — Text "STOP" from your cell to the Surge long code

Within 1–2 seconds:

- Twilio auto-replies with your custom Surge confirmation copy — no code on your side.
- A red-bordered row appears in the Inbound Stream: "Advanced Opt-Out · STOP".
- The activity log narrates the OptOutType=STOP event with a link to the Twilio doc.
- Twilio adds your cell to its block list.

Say: "That reply, that block-list entry, that audit record — all handled by Twilio's Advanced Opt-Out feature on the Messaging Service. You configured the reply copy once in Console. No downstream sync, no code, no risk of missing a STOP."

STEP 2 PART A — Prove the block by trying to send again

Click the Use Case 1 tab. Click "Send now". The activity log will show Twilio accepted the request but the response will surface error 21610 — attempt to send to unsubscribed recipient — and the Delivered counter for your cell does not increment.

Say: "Continental's STOP compliance is now enforced at the Twilio edge. Even a rogue downstream send is blocked before it leaves. That's a compliance posture your incumbent can't match without your engineers writing the enforcement themselves."

STEP 3 PART A — Text "START" to re-opt

Back on Use Case 3. Text START from your cell. Twilio auto-replies with the re-opt confirmation. Row appears: "Advanced Opt-Out · START". Number is removed from Twilio's block list. Future sends go through.

Say: "Same automation for opt-in. Same audit trail. Same zero code."

Use Case 3 · continued

STEP 4 PART B — Text "please stop bugging me" from your cell

This is NOT a keyword. Twilio doesn't classify it. Twilio delivers the raw text to our inbound webhook. An amber-bordered row appears in the Inbound Stream: "Free-form (needs human review)".

Say: "Every SMS platform has this problem: cardholders don't always say STOP. They say "quit texting me," or "I paid this last week," or "wrong number." Your incumbent misses these because it only classifies the exact word STOP. Watch what Twilio does."

STEP 5 PART B — Click "Add to DNC" on that row

The row moves. The Do-Not-Contact panel on the right lists that number. Timeline in the activity log stamps the addition.

Say: "That number is now on our app-side Do-Not-Contact list. This is a suppression layer that Continental controls — separate from Twilio's Advanced Opt-Out block list. Two important reasons."

One: it works across every sender you ever add. If you buy a second Surge number tomorrow, the DNC still applies. That's the cross-sender governance the RFP explicitly asks for. Two: it lets a compliance officer intervene proactively — for example, adding a number after a support call, or a complaint routed through Genesys.

STEP 6 PART B — Prove the DNC block by trying to send again

Click the Use Case 1 tab. Click "Send now". The response payload will show "blockedByDnc": [<your cell>]. The send never leaves our server — Twilio isn't even called for that number.

Say: "That's defense in depth. Twilio blocks at the edge for STOP. We block before the API call for our own DNC list. Cardholders don't receive messages they asked us to stop."

Honest concession on re-opt: Twilio's Advanced Opt-Out block list is Console-managed. There is no public REST API to remove a number from that block list — the customer must text START. If Continental wants programmatic re-opt for support-desk workflows, that's a Support case or a manual Console action.

RFP use case 4 · Business-user administration (talk-track)

No live UI. This is the section you present with slides on the projector when the evaluation team asks about business-user self-service.

BUSINESS USERS SELF-SERVE FOR

- DNC / suppression list management via Messaging > Advanced Opt-Out (Console).
- Opt-out confirmation wording — the custom Surge copy is editable in Console without a code deploy.
- Sender Pool / Messaging Service configuration — add or remove numbers, change routing rules.
- Scheduling adjustments and pausing campaigns — Programmable Messaging scheduling window from 15 min to 7 days out.
- Delivery monitoring, alerts, reports — Console + Insights + Event Streams for BI destinations.
- User & role management via Twilio IAM for tenant-scoped access.

WHERE WE CONCEDE HONESTLY

Changing SMS body copy itself still requires an application code change. Twilio does not ship a business-user editor for SMS body text.

If Continental Finance wants a business-user-facing content editor on top of Programmable Messaging, that is a scoped services engagement — quoted separately. We won't fake it in this demo.

Say: "The RFP explicitly asks for business-user self-service without IT code changes. For everything except message body copy, Console + IAM cover it. For message body copy, you get a services engagement price with a real Twilio partner. That is honest, and if that gap is the deal-breaker, we'll size the services engagement in the follow-up."

Honest concessions summary

You will get asked. Answer directly.

BUSINESS-USER SMS BODY EDITOR

Twilio does not ship this. Console covers DNC, sender config, opt-out wording, scheduling, monitoring — but NOT body copy. Body copy edits are a code change, or a services engagement to build a content editor on top of Programmable Messaging.

PROGRAMMATIC RE-OPT FROM TWILIO BLOCK LIST

Advanced Opt-Out block list is Console-managed. No public REST API to remove numbers from that block list. Customer texts START, or Support intervenes.

GENESYS INTEGRATION

Continental's contact center is entirely Genesys. Twilio is not competing to replace it. If agents ever need SMS visibility inside Genesys, that's a services integration — not part of this demo.

Appendix · Suggested Advanced Opt-Out copy

STOP confirmation

You've been unsubscribed from Surge Mastercard reminders.
Reply START to resubscribe. For help, reply HELP.

START confirmation

You're re-subscribed to Surge Mastercard reminders.
Msg&data rates may apply. Reply STOP to opt out.

HELP reply

Surge Mastercard from Continental Finance.
Call 1-877-xxx-xxxx for cardholder services.
Msg&data rates may apply. Reply STOP to opt out.

Appendix · Sample Programmable Messaging payloads

Immediate send

```
POST /2010-04-01/Accounts/{AccountSid}/Messages.json
```

```
messagingServiceSid = MG...  
to = +15551234567  
body = "Hi Jane, this is a reminder that your Surge Mastercard ending in 4832 has a  
payment of $47.50 due on Sep 15. Reply STOP to opt out."
```

Scheduled send (15 min – 7 days out)

```
POST /2010-04-01/Accounts/{AccountSid}/Messages.json
```

```
messagingServiceSid = MG...  
to = +15551234567  
body = "Hi Jane, this is a reminder..."  
scheduleType = fixed  
sendAt = 2026-09-12T14:00:00Z
```

Returns MessageSid + status=scheduled.

Cancel a scheduled message

```
POST /2010-04-01/Accounts/{AccountSid}/Messages/{MessageSid}.json
```

Status = canceled

Returns status=canceled. Only valid while message is in scheduled status; returns error 30409 otherwise.